KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

Project-based Practical Report on

Bike / Cycle Rental Booking System - Phase 4

Submitted in partial fulfilment of the requirements for
"Practical Course Title" for the 1st Semester

Master of Computer Applications (MCA)

Submitted by

<Candidate Name> ***<Candidate Reg. no.>***

Under the guidance of

<Guide Name>

Contents

No.	Contents	Page
1	Title of the Project	3
2	Introduction	4
3	Objective of the Project	7
4	Scope of the Project	9
5	Software Specifications	11
6	Workflow of the Project	14
7	Modules of the Project - Phase-wise explanation of design and execution of Modules	20
8	Source Code (Only functional code structure)	33
9	Features of the Project	35
10	Future Enhancements	44
11	Conclusion	45

Project at a glance

Purpose	Demonstrate a Java web workflow for cycle discovery and short-term rental booking.
Runtime	Java 17 target, Servlet 4.0 APIs, JSP, and Apache Tomcat 9.
Rider journey	Register, sign in, browse, book, monitor a countdown, cancel, and log out.
Fleet	Six seeded cycles with hourly rates from Rs. 25 to Rs. 60.
State	Accounts, bookings, activity, and availability are held in application memory.
API	GET /api/state; POST /api/book; POST /api/cancel.
Access	Authenticated pages and API operations use an HttpSession with a 30-minute timeout.
Boundary	No database, password hashing, payment, or administrative inventory screen is implemented.

1. Title of the Project

The project is titled Bike / Cycle Rental Booking System - Phase 4. It is a Java web application that brings together an account model, cycle catalogue, booking service, browser interface, and Servlet-based server integration. A rider can register, log in, inspect available cycles, choose a rental duration, make a booking, follow its countdown, and cancel a booking that belongs to the rider. The title describes the implemented application rather than a proposal for a commercial fleet business.

The project is presented as an academic practical with a defined implementation boundary. Its most important learning outcome is the connection between a user action in a browser and a state change in Java on the server. JSP pages provide the application views; Servlets receive requests; service classes enforce business rules; model classes represent users, cycles, bookings, and activity; and JavaScript updates the live dashboard. Apache Tomcat 9 hosts the application and supplies the Servlet/JSP runtime.

The application is deliberately small enough to inspect as one project. Its six cycles and in-memory services make the booking lifecycle visible without requiring a separate database installation. That same design also creates a clear limitation: users and bookings do not survive a Tomcat restart. The report therefore describes both what the source implements and what it does not implement, distinguishing demonstrated behavior from possible later improvements.

The name is used consistently across the landing page, navigation, and report so the implemented product is immediately identifiable. ?Booking system?

Project boundary and report reading guide

This report follows the supplied practical-project template and retains its eleven main headings. The material beneath each heading is expanded to document the application in detail: its purpose, objectives, scope, software requirements, workflows, phase-wise modules, source structure, features, possible enhancements, and conclusion. Supporting diagrams and the supplied application screenshots are included under the workflow, module, and feature headings rather than as unrelated report chapters.

The system is described as it exists in the source files. For example, the report uses the term in-memory collection for users and bookings because the project contains no JDBC connection, SQL schema, or database driver. It calls a booking active only while the booking remains in the running process and its end time has not passed. These definitions prevent the report from implying persistence or payment functions that the project does not contain.

When reading the phase discussion, Phase 1 refers to the Java domain model, Phase 2 to service abstractions and validation, Phase 3 to the browser interface, and Phase 4 to Servlet/JSP integration and deployment. These labels explain how the pieces fit together; they do not claim that separate archived submissions for each phase are included in this report package.

The report follows the supplied eleven-part academic outline while expanding the workflow and module sections to explain the code path in greater detail. Diagrams are explanatory views of the existing implementation, not separate design specifications.

2. Introduction

Cycle rental is a short-duration service: a rider needs to know what is available, how much it costs, whether another rider has already reserved it, and how long a current reservation remains active. A web application can put these questions in one place. It can show the catalogue and let the rider submit a request without depending on a separate manual update for every availability change.

The Bike / Cycle Rental Booking System models this interaction as a small web application. A visitor creates an account through a registration form. After server-side account validation, the visitor signs in. The authenticated application presents cycle cards, filters by cycle type, displays hourly rates, accepts a selected duration, and provides booking and cancellation actions. A separate bookings view presents active rental details and a countdown. A profile view displays account and session details already associated with the login.

The application demonstrates a server-authoritative flow. A browser can check form fields and preview a cost, but it does not decide whether a cycle is available. The Servlet passes the request to a service, and the service checks the relevant state before changing it. The browser then displays a response and refreshes the state it received. This separation is the central integration idea documented in this practical project.

The application is intentionally small enough for a learner to trace a complete request without a framework concealing the Servlet lifecycle.

Why a shared availability view matters

An availability display is useful only when the view and the booking action agree. If a browser used an old list and treated that list as final, two riders could both believe they had reserved the same cycle. This project reduces that problem by sending a booking request to the server and checking the current cycle state inside the synchronized booking service. A cycle already marked unavailable is rejected with a booking error rather than recorded a second time.

The interface also combines information that would otherwise be scattered across pages. Cycle type and rate appear on each card; duration and estimated total are placed with the booking control; active bookings show the selected cycle, hours, total, time remaining, and cancel action; and the activity panel provides recent events. The rider can therefore move between discovery and booking management while the shared navigation keeps the application areas visible.

The implementation remains an educational, single-process solution. A synchronized method protects state transitions among requests handled by one application instance, but there is no distributed lock or persistent reservation store. A restart recreates the initial fleet and loses the registered accounts and active rentals. These boundaries are important when interpreting both the system’s benefits and the next design steps.

A shared state is useful only when the server makes the final decision. Two browser tabs may display the same cycle as available, so the booking operation must re-check availability when each request arrives.

Project problem statement

The practical problem addressed is the need for a simple, consistent flow from cycle discovery to rental booking. A rider needs a clear sign-in route, an understandable list of cycles, an availability indicator, a duration choice, a cost preview, and a way to monitor or cancel an active booking. The application joins these elements into a browser-based experience backed by Java request handlers and services.

There is also a data-integrity problem. Browser input is not trusted simply because the project includes JavaScript validation. A request can be altered or sent without using the intended page. Registration fields must therefore be checked by the server, and a booking must be checked for a valid user, an allowed duration, a cycle that exists, and a cycle that remains available. A cancellation must be checked against the owner of the booking.

The project responds to these requirements within an intentionally modest implementation. It supplies a fixed fleet and temporary account and booking storage so the complete flow can be demonstrated. It does not attempt to provide a production identity system, long-term record keeping, a staff interface, a payment gateway, or a scalable multi-server deployment. The problem scope is an integration exercise, not a claim that every commercial rental requirement has been solved.

The practical problem is to connect familiar rider tasks to explicit server behavior: creating an account, establishing a session, selecting inventory, and changing a reservation. The project makes these transitions observable through page messages, availability counts, booking rows, and activity entries.

Users, roles, and system boundary

The primary actor is a rider. A visitor can open the public registration and login views. A registered rider can authenticate, use the protected cycle catalogue, inspect their own active bookings, see profile information, and log out. The implementation does not define an administrator or employee role. Cycle inventory is seeded by the booking service rather than edited through an administrative screen.

The browser, web application, and runtime form the system boundary described here. The browser renders JSP-generated pages and runs the supplied JavaScript. Tomcat maps URLs to Servlets and renders JSPs. Servlet code obtains the application services created at startup and forwards request data to them. The service layer applies business rules to model objects held in memory. JSON responses carry the state required by the live dashboard.

External systems are limited. The project may load web fonts from a public font service when network access permits, but the application behavior is not based on a remote booking API. No database or email provider is configured. The Tomcat installation and Java Development Kit are deployment prerequisites. Treating these elements as boundaries clarifies what must be available to run the demonstration and which capabilities require future integration.

A visitor can reach the public login and registration routes; an authenticated rider can open the cycle, booking, and profile pages and use the booking API. The application does not implement staff accounts or an administrator role.

Operating assumptions and terminology

This report uses “cycle” and “bike” for the same rental item because both terms occur in the supplied interface and project name. A cycle has an identifier, display name, type, hourly rate, and availability flag. A booking links one user to one cycle for a number of hours. The total shown by the application is calculated from the hourly rate multiplied by the selected duration; there is no separate tax or payment calculation in the source.

The application is assumed to be running as one Tomcat web application instance. Its services are shared within that instance by Servlets through the ServletContext. The service data lasts only as long as that instance remains running. The demonstration uses a browser that can submit ordinary HTML forms and JavaScript fetch requests. A valid browser session is represented by an HttpSession and the user object stored in it.

Screenshots are treated as supplied user-interface evidence. Their account values and on-screen availability reflect the captured demonstration state, not permanent project data or an independently verified account record. Diagrams are implementation-oriented views of the application. In particular, the conceptual relationship diagram is not a deployed relational database schema. These assumptions keep the report precise and avoid turning an illustration into a claim about a feature that is absent from the source.

?Cycle? and ?bike? refer to the same seeded rental item, while ?booking? and ?reservation? refer to a time-limited allocation. A booking has a rider, cycle, duration, total, and end time; an available cycle has no current active booking.

3. Objective of the Project

The primary objective is to demonstrate how a Java model and service layer can be integrated with a browser-based application through Servlets and JSP. The project makes this objective visible through a complete rider journey: registration, server-side validation, authentication, session-protected pages, cycle discovery, booking, booking status display, cancellation, and logout.

The objective is not merely to display static HTML. A successful booking changes availability in service state. A subsequent response reports the resulting state, and the browser redraws the relevant cards and booking information. Registration creates a user object held by the user service, while login establishes session state associated with the user. Each stage therefore demonstrates a different part of a request-to-response path.

The project also aims to keep responsibilities understandable. Domain classes represent data; service interfaces define operations; implementations own the collection-backed rules; Servlets translate HTTP requests and responses; utility classes handle authentication lookup and JSON construction; and JSP/JavaScript files present the user experience. The separation is small enough for a practical assignment but clear enough to explain how later persistence or testing could be added.

The objective is to demonstrate how Java model and service code can be integrated with a browser interface through JSP, Servlets, and an HTTP session.

Functional objectives

The functional objectives begin with account handling. A visitor should be able to submit a full name, username, email address, phone number, password, and password confirmation. The server checks missing data and format rules, rejects a duplicate username, and reports a useful error to the registration view. On successful registration, the browser is redirected to login with a registration notice.

A second objective is authenticated access. Valid credentials establish a fresh session with a thirty-minute inactivity timeout. The cycle, bookings, and profile pages use the authenticated user. A request for dashboard state or a booking operation without a valid user receives a logged-out response or a redirect. Logout invalidates the session rather than merely hiding a navigation element.

The booking objectives are to list the fixed cycle fleet, filter by category, preview hourly cost, book an available cycle for a chosen duration, show the active booking and its countdown, and allow the owner to cancel. Recent activity should reflect booking changes while displaying other participants generically as “Another rider.” These objectives are implemented by the interaction among the booking service, JSON state builder, API Servlet, and dashboard scripts.

The rider flow is organized around clear outcomes rather than a long form sequence. Registration validates required identity and contact fields, login establishes a session, the catalogue shows cycle type and hourly price, and duration selection previews the total.

Reliability and learning objectives

The reliability objective is to repeat important validation at the server boundary. JavaScript improves feedback but is not the final authority. Service checks reject unsupported durations, missing cycles, already-booked cycles, missing users, unknown booking identifiers, and cancellation attempts by a different user. Synchronized booking operations protect shared state changes within one running Tomcat process.

The project also demonstrates practical use of exceptions. Invalid user data, authentication failures, and booking conflicts have distinct exception types. Servlets catch the expected domain exceptions and turn them into redirects, page messages, or JSON responses with suitable status codes. This provides a concrete example of keeping business failures separate from the browser’s rendering logic.

As a course exercise, the implementation objectives include using Java packages and interfaces, configuring a Servlet 4.0-compatible application, deploying to Tomcat 9, and documenting where the prototype stops. It is important to state the limitations alongside the successful flow: no data persistence, password hashing, payment, administrative inventory editing, automated test suite, or distributed consistency is included.

Reliability in this exercise means predictable decisions at the service boundary: invalid values are rejected, unavailable cycles are not double-booked within the instance, and a rider cannot cancel another user?s booking.

4. Scope of the Project

The implemented scope begins with two public entry views: account registration and login. Registration accepts the profile fields represented by the User model and validates them on the server. The login service checks the username and password against users held in memory. Successful login creates an authenticated session; logout invalidates that session. The protected area includes the cycle catalogue, bookings view, and profile view.

The fleet scope is a seeded list of six cycles. Each item has a fixed identifier, name, category, and hourly rate. The available categories represented by the source are Mountain, Hybrid, Road, and Electric. The dashboard displays cycle cards and availability, offers category filters, accepts a duration selection, previews the cost, and submits a booking action to the server.

The booking scope covers one active reservation per cycle state transition, ownership-aware cancellation, calculation of total cost, time remaining, and release after a booking end time is reached. The dashboard offers one-to-eight-hour choices, while the service accepts a maximum of twelve hours when called directly. The JSON API supports current state, booking, and cancellation. The report records this UI-to-service limit difference rather than treating the two validation boundaries as identical.

The implemented scope is a single rider-facing web application with public account entry, authenticated cycle discovery, short-duration bookings, a live activity view, and session details. Shared JSP fragments and SVG cycle illustrations keep common navigation and imagery consistent.

Protected pages and user information

The profile page presents account fields available from the authenticated User object and displays session-related information. It is a read-only view in the inspected implementation. It does not let a rider edit personal information, change a password, delete an account, or recover an account. The report treats those omissions as out of scope rather than implying the profile screen provides account management.

The bookings view is centered on active bookings. It shows a booking identifier, cycle, duration, total, remaining time, progress indicator, and a cancel action. When there is no active booking, the empty state invites the rider to browse cycles. The activity panel displays a bounded set of recent booking events. State is refreshed while the browser page is active, and countdown text changes locally every second between server updates.

The pages share navigation and JSP fragments so that the same links and layout conventions are used across the authenticated area. Client-side modules separate view rendering from event handling. The report documents these as presentation and interaction features, but not as a substitute for a broader accessibility audit or a cross-browser certification exercise.

The guard fragment reads the current session and redirects unauthenticated requests to login, so protection is applied when a page is requested rather than only when a navigation link is hidden. The profile displays account attributes and session timing, but does not offer editing.

Explicit exclusions

Persistence is excluded. Users, bookings, activity, and availability live in Java collections managed by the running application. A restart reconstructs the six seeded cycles and empty account and booking stores. The project contains no database schema, SQL statements, JDBC connection, ORM, DAO, or migration tool. A conceptual relationship drawing can explain object links but must not be interpreted as implemented tables.

Commercial rental functions are excluded. There is no payment collection, deposit handling, refund, tax, invoice, booking history archive, asset maintenance status, or external identity provider. The project does not contain a staff or administrator account, a fleet editor, route planning, map integration, e-mail/SMS notification, or inventory import. The fixed fleet is changed in source rather than through an application screen.

Production deployment protections are also outside scope. The project does not store salted password hashes and does not provide production HTTPS configuration, account recovery, a complete CSRF-token mechanism, rate limiting, audit retention, or multi-instance coordination. The X-Requested-With header check on API mutation requests is an application request guard; it is not described as a complete substitute for a security design.

No database driver, schema, DAO, SQL statement, payment gateway, email confirmation, password reset, inventory editor, or analytics service is present in the source package. User and booking collections are recreated when the application starts, and the fleet is seeded in Java.

Constraints and operational assumptions

The build script targets Windows paths and a Tomcat 9 installation. The configured Tomcat folder must contain the Servlet API JAR and be writable for deployment. Java source is compiled with the Java 17 release target, so the selected JDK must support that target. Tomcat 10 and later use the Jakarta namespace and are not drop-in substitutes for this project's javax.servlet imports.

The single application instance is the consistency boundary. Synchronized methods prevent two request threads from interleaving certain booking changes in that instance; they cannot coordinate separate Tomcat processes. The browser's four-second polling interval is a display refresh strategy, not an event-streaming guarantee. The server remains the source of truth when an operation is submitted.

The demonstration assumes a modern browser with JavaScript enabled for the live interactions. The login and registration pages remain HTML form views, and the server repeats essential checks, but the dashboard relies on JavaScript fetch calls for its dynamic state. External font loading is optional visual enhancement and should not be confused with a runtime dependency for service behavior.

The supplied build script assumes a Windows environment, a Java compiler compatible with the configured release target, and a local Tomcat 9 installation whose Servlet API jar is available. The project is deployed under a fixed application context.

5. Software Specifications

The application source is written in Java and compiled using the Java 17 language/API release target specified by build.bat. A Java Development Kit is required to run javac. The web application targets Apache Tomcat 9 and declares a Servlet 4.0 deployment descriptor. Source imports use javax.servlet; that package namespace is material to the server version choice.

Tomcat provides the Servlet container and JSP runtime. At application startup, the AppListener constructs UserOperationsImpl and BookingOperationsImpl and stores them in the ServletContext. Servlets retrieve those shared service objects through the application context, so one set of in-memory service collections is used by requests within the web application instance.

The application context is BikeRental in the supplied deployment instructions. The welcome file is login.jsp. The build script compiles the Java packages into WEB-INF/classes and copies the webapp directory to Tomcat's webapps/BikeRental directory. A successful compile alone does not establish that Tomcat started or that the browser flow was verified; those are separate stages in the operational process.

The specification is intentionally grounded in files present in the project: Java sources define the model, services, and Servlets; JSP and shared fragments define pages; JavaScript manages validation and live interaction; CSS defines the shared presentation.

Technology stack

The presentation layer uses JSP, HTML, and CSS. JSP pages render login, registration, dashboard, bookings, and profile experiences. Shared JSP fragments hold common page pieces. CSS defines the application layout, cards, buttons, status indicators, activity components, and responsive behavior. The interface uses an external font stylesheet when a network connection is available, while system behavior is provided by local application code.

JavaScript provides field feedback and live interaction. validation.js supplies browser-side validation helpers; register.js and login.js connect those checks to their forms; dashboard-view.js contains functions that render values from a state object; and dashboard.js handles polling, event listeners, booking requests, cancellation, countdowns, and status messages. The view module escapes dynamic text before inserting it into HTML.

The HTTP integration uses ordinary form submissions for registration and login and fetch requests for dashboard actions. The API returns JSON for state, booking, cancellation, and expected booking errors. The response includes no-store cache behavior and UTF-8 encoding. The source does not introduce a third-party Java framework, database library, ORM, or build system such as Maven or Gradle.

The Java layer targets Java 17 and the javax.servlet API provided by Tomcat 9. JSP templates render server-side account and page data, while browser JavaScript uses fetch for the live JSON endpoints.

Data objects and sample fleet

The User model stores a user ID, full name, username, password, e-mail, and phone number. Cycle stores an identifier, display name, category, hourly rate, and availability flag. Booking associates a user with a cycle and records the duration and booking date; its calculateCost method multiplies the cycle rate by duration and safely returns zero when required values are missing. Activity records a timestamp, username, and message as immutable fields.

ID	Cycle	Type	Rate / hour
C201	Trek Mountain Explorer	Mountain	Rs. 40
C202	Hero Cityline	Hybrid	Rs. 25
C203	Firefox Road Runner	Road	Rs. 35
C204	Hercules Trailblazer	Mountain	Rs. 45
C205	Atlas Urban Glide	Hybrid	Rs. 30
C206	Volt Electric Cruiser	Electric	Rs. 60

BookingOperationsImpl seeds six cycle objects. The source values are C201 Trek Mountain Explorer at Rs. 40 per hour, C202 Hero Cityline at Rs. 25, C203 Firefox Road Runner at Rs. 35, C204 Hercules Trailblazer at Rs. 45, C205 Atlas Urban Glide at Rs. 30, and C206 Volt Electric Cruiser at Rs. 60. The available type labels are Mountain, Hybrid, Road, and Electric.

These values are starting data, not a catalogue loaded from persistent storage. The booking service owns a cycle list, booking list, end-time map, bounded activity list, and monotonically incremented booking identifier counter. The user service uses a ConcurrentHashMap keyed by username. A Tomcat restart recreates these structures from constructors and loses runtime changes.

User holds the account identity and contact attributes used by registration, login, and profile display. Cycle represents a seeded item and its type and hourly rate; Booking associates a rider and cycle with a duration, calculated total, and end timestamp; Activity records a short event for the feed.

Build and deployment requirements

The supplied build.bat begins by checking for servlet-api.jar inside the configured Tomcat lib directory. It changes the working directory to the project root, compiles model, exception, service, utility, and Servlet source files with UTF-8 encoding and the Java 17 release target, and writes compiled classes under webapp/WEB-INF/classes. Compilation errors stop the script before the deployment copy proceeds.

For deployment, the script removes the existing BikeRental webapps directory if present and copies the project webapp tree to that target. The configured Tomcat path is machine-specific, so the operator must confirm TOMCAT_HOME before running it. Because the script replaces the target deployment directory, the operator should understand that behavior and stop or restart Tomcat in a way that permits a clean redeploy.

The operational sequence is: verify java and javac, configure the JDK and Tomcat, set the Tomcat path in build.bat, execute the build, start Tomcat, and open the application context in a browser. If a request returns 404, investigate deployment path and Tomcat logs. If javac reports that release 17 is unsupported, the JDK is too old for the specified release target. These checks distinguish build failures from container and browser failures.

A successful compilation confirms that the Java sources can be compiled against the selected

Servlet API; it does not confirm that Tomcat has started or that the browser can reach the deployed context.

System quality requirements and constraints

The application has a modest usability requirement: users should be able to identify available cycles, compare category and hourly price, choose a duration, and understand the result of a booking or cancellation. The screenshots show the navigation, status signal, empty state, active booking row, and profile view that support these tasks. Registration fields include labels and inline validation messages; the dashboard uses status messages and live updates.

Correctness requirements are stronger than visual feedback. The server must reject invalid account data, invalid credentials, unavailable cycle requests, and unauthorized cancellation. State transitions should be atomic within the synchronized service methods. API responses should make logged-out and booking-conflict cases distinguishable from success so the browser can display appropriate messages and refresh state.

Operational constraints include volatile data, one runtime instance, and a Java/Tomcat compatibility requirement. Security constraints include plaintext password storage and the absence of a full production security control set. Performance is not benchmarked in the supplied project; six seeded cycles and a bounded activity list are small in-memory demonstration data, not evidence of load capacity. Those facts guide accurate quality claims.

The interface should expose clear labels, keyboard focus, readable status feedback, and usable controls at narrow widths. Correctness is more important than a client-side preview: the server validates identity, duration, availability, session, and booking ownership. Maintainability depends on preserving these boundaries when a page or API changes.

6. Workflow of the Project

The end-to-end request path begins when a visitor opens a public JSP page or an authenticated rider activates a dashboard control. The browser submits form fields or a fetch request. Tomcat maps the URL and HTTP method to a Servlet. The Servlet reads parameters, obtains the relevant service from the ServletContext, performs request-level checks, and delegates a business operation. A service either changes in-memory state or raises a domain exception.

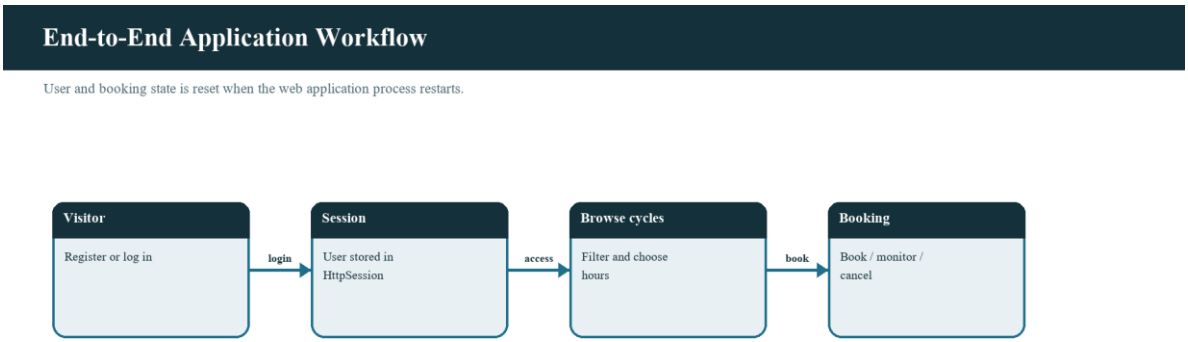


Figure 1. Application workflow from browser action through Servlet and service to the response.

The Servlet then translates the outcome into an HTTP response: a redirect or forwarded page for account forms, or JSON for API operations. The browser follows a redirect, displays a server-provided error or notice, or applies a returned state snapshot to the dashboard. This layered sequence separates transport decisions from service rules and visual rendering.

The diagram summarizes the application as it is implemented. It does not show a database, payment gateway, or external booking service because none is included. On every request, the Servlet container also supplies request, response, session, and application context objects. These runtime objects are the bridge between page interaction and shared application services.

The workflow follows one request lifecycle from the browser into the application and back. Public account requests use form posts and redirects; authenticated booking operations use same-origin JSON requests and return updated state. Each stage has a visible result, but the service operation remains authoritative.

Registration workflow

Registration starts on register.jsp, which presents full name, username, e-mail, phone, password, and password confirmation fields. Browser scripts provide immediate feedback for missing or malformed values, but the form posts to RegisterServlet, which repeats the authoritative checks. Text fields are trimmed before the User object is constructed; password values are compared but are not trimmed by the Servlet.

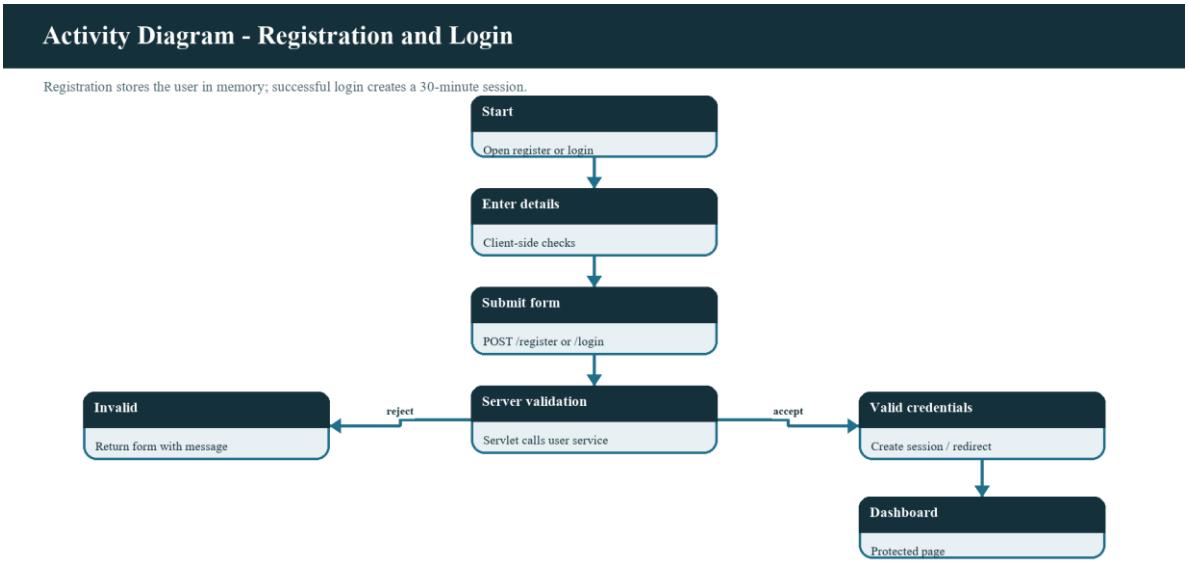


Figure 2. Registration and server validation activity.

RegisterServlet assigns a generated user ID and calls UserOperations.registerUser. The implementation checks null user data, name format, username format, password minimum and composition, e-mail form, ten-digit phone number, and username duplication. A valid account is inserted into the service map. The Servlet redirects to login.jsp with a registration indicator so the user sees the success notice after the POST.

If the server rejects data, the exception message is attached to the request and register.jsp is forwarded. This preserves a clear distinction between client-side guidance and server-side enforcement. The following activity diagram describes the path; it is a workflow view rather than a separate automated-test result.

Account data validation sequence

Validation is performed in layers so a malformed request cannot become a registered account merely because it bypassed JavaScript. The browser can explain expected formats before submission. The Servlet normalizes text fields, checks that password and confirmation match, constructs the object, and delegates to the service. The service is the single place where the user rules are applied to the stored account operation.

The username is trimmed and must contain at least four letters, digits, or underscores. The password must have at least six characters and include at least one letter and one digit. The full name and e-mail use regular expressions in the service, and the phone number must be exactly ten numeric digits after trimming. A username already present in the map is rejected. Error messages are returned through a registration-page forward rather than being represented as a successful redirect.

These checks define a project-level validation policy, not comprehensive identity proofing. The e-mail check does not send a verification link, the phone check does not send a code, and the password rule is not a replacement for a modern password-hashing strategy. A production extension would preserve server validation while adding verification, secure credential storage, and recovery controls.

Validation is layered to improve feedback without trusting the browser. Client scripts can identify an empty field or a malformed value close to the control, while the Servlet and service repeat the checks on submitted data. The server also controls uniqueness and assigns the user identifier.

Login and session workflow

Login is handled by LoginServlet. A GET request checks for an existing session without creating a new one; if the session contains a user, it redirects to the dashboard, otherwise it redirects to the login page. A POST reads the username and password, rejects missing values, and delegates credential comparison to UserOperations.authenticateUser.

On successful authentication, any prior session is invalidated and a fresh HttpSession is created. The user object is stored under the session key user, and the inactivity interval is set to thirty minutes. The web.xml session configuration also specifies a thirty-minute timeout and an HttpOnly cookie. A protected JSP can use the Auth helper to obtain the current user without creating an empty session.

This gives the application a clear boundary between public pages and authenticated pages. API endpoints also use the current session user; a missing user results in an unauthenticated JSON response. Logout invalidates the session. These controls demonstrate session lifecycle management, while leaving production matters such as secure-cookie configuration over HTTPS and complete cross-site request protections for further work.

A successful login creates a fresh HttpSession and stores the authenticated user for later requests. Protected JSPs obtain that user through the shared guard, and the dashboard API uses the same session boundary. The configured inactivity timeout limits how long an idle session remains valid; logout invalidates it.

Cycle booking activity

A booking begins when an authenticated rider chooses a cycle and a duration on the dashboard. The JavaScript reads the cycle identifier and selected hours and posts them as URL-encoded form data to api/book. The request includes the XMLHttpRequest header used by the API mutation guard and same-origin credentials so the browser sends its session cookie.

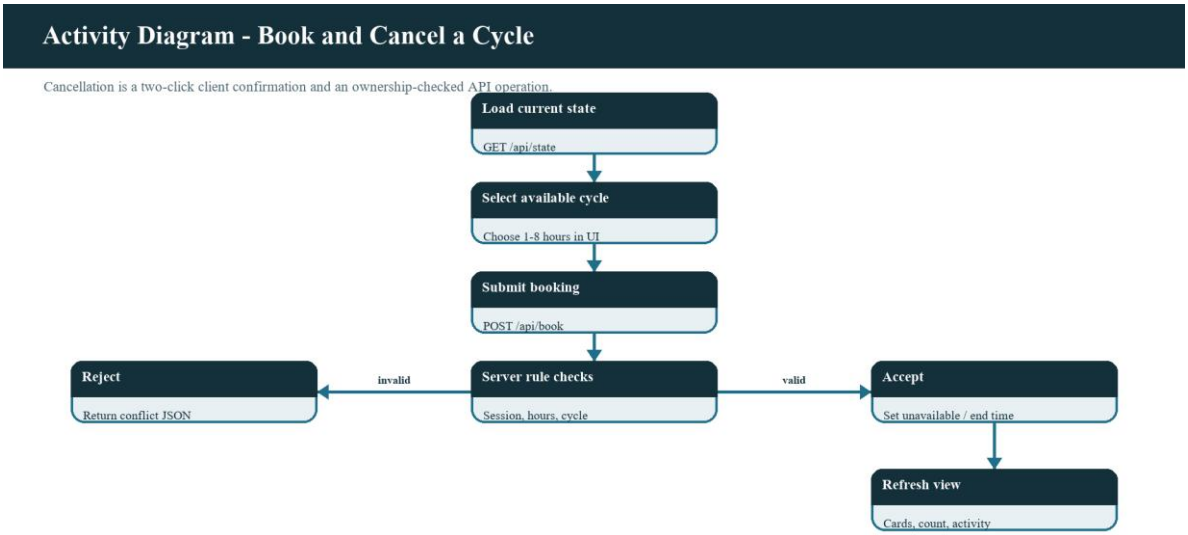


Figure 3. Booking and cancellation state transitions represented in the service.

ApiServlet checks the request header and session before parsing the duration. BookingOperationsImpl releases expired entries, checks that the user exists, verifies that the duration is between one and twelve hours, looks up the cycle, and rejects an unknown or unavailable item. The synchronized service marks the cycle unavailable, creates a booking identifier and date, records an end time, adds an activity entry, and returns the booking to the Servlet.

The Servlet calculates the displayed total from the booking and returns a JSON response containing a message and a new state snapshot. The dashboard applies the new snapshot, updates availability and booking count, and shows a toast. The interface presents duration choices from one to eight hours, which is narrower than the service maximum; the server remains the authority for requests regardless of the visible selection.

Booking API request sequence

The sequence diagram expands one booking request across participants. The rider selects a cycle and hours in the browser. dashboard.js encodes those values and calls the API. Tomcat routes the path to ApiServlet. The Servlet retrieves the session user, decodes the duration, obtains BookingOperations from the application context, and invokes bookCycle.

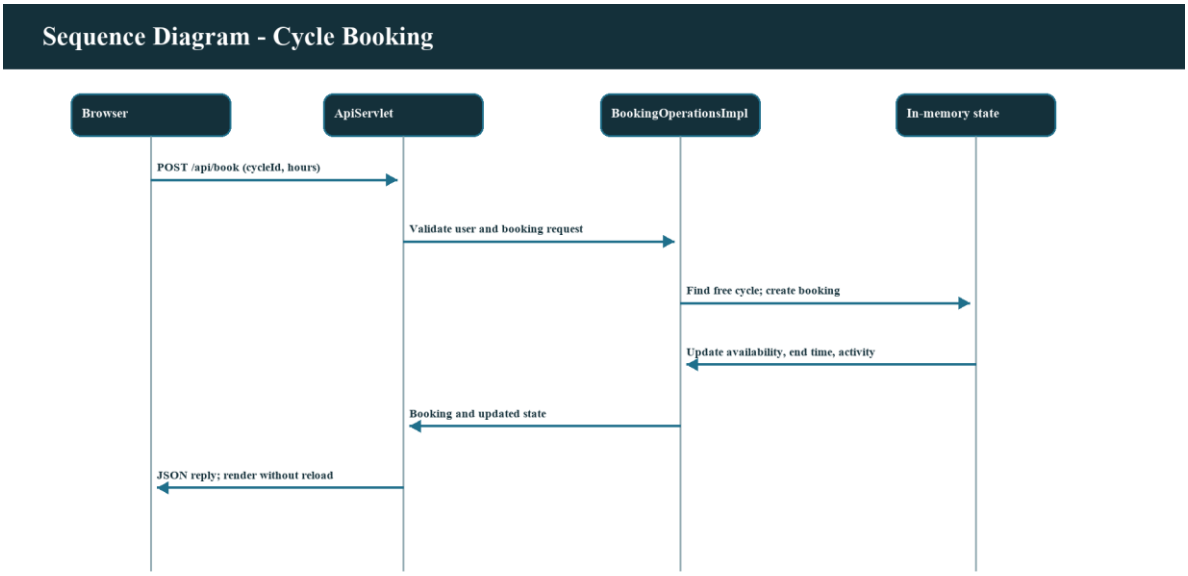


Figure 4. Browser-to-server request sequence for a cycle booking.

The service validates the domain request before changing the cycle list, booking list, end-time map, and activity list. If the cycle is unavailable or the request is invalid, BookingException returns control to the Servlet and a conflict-style JSON reply is sent. If the request succeeds, the Servlet forms a user-facing message and constructs the refreshed state with StateJson. The browser applies that state and redraws only the components whose content changed.

The cycle state change and booking creation occur inside a synchronized operation, preventing two request threads in the same service instance from both passing the same availability test. The browser's cost preview is not a security check; the service determines the actual cycle and duration and the server response reports the resulting amount. No payment authorization is performed.

Cancellation and booking expiry

Cancellation is initiated from the rider's active bookings view. The interface requires a second click within a short confirmation window, reducing accidental cancellation from a single click. That confirmation is a usability interaction, not an authorization boundary: the service still checks that the current user owns the booking before it changes the booking list.

BookingOperationsImpl searches the active bookings for the requested identifier. If the booking belongs to another username, it throws a booking exception. If the owner matches, the service marks the associated cycle available, removes the end-time record, deletes the booking, and records a cancellation event. If the booking identifier is not present, the operation returns a not-found style booking error instead of silently reporting success.

Expiry is lazy rather than driven by a background scheduler. Operations that read cycle lists, rider bookings, or recent activity call releaseFinishedBookings. When the stored end time is no longer in the future, the service removes the booking, restores availability, removes the timer entry, and records a finished-ride event. The countdown in the browser provides visual feedback; the service state is authoritative.

Cancellation is presented as a two-step confirmation in the interface to reduce accidental clicks, but the service still verifies that the current rider owns the target booking. Expiry is lazy: a later service operation evaluates end timestamps and releases elapsed bookings.

Failure handling and HTTP outcomes

Expected failures are part of the application workflow. Invalid registration is reported on the registration page, invalid credentials are displayed on login, unauthenticated API access returns an authentication response, an unsupported API path returns not found, and booking-rule conflicts return an operation error. These outcomes help the browser distinguish a rejected action from a successful state change.

ApiServlet sets the response status, disables caching with Cache-Control: no-store, specifies JSON content type, and uses UTF-8 encoding. The mutation path checks the expected request header before accessing service state. A logged-out reply includes a flag that dashboard.js uses to send the browser to the login page with a required-login notice. A normal booking reply includes the updated state used to refresh the dashboard.

The interface also handles network failure. If fetch cannot reach the server, the dashboard displays a message, changes the live indicator to reconnecting, and re-enables controls for a retry. This is a useful recovery path for transient errors, but it does not claim that an operation which timed out definitely did not reach the server. A more advanced system would use idempotency keys and explicit retry semantics for ambiguous outcomes.

Expected business failures should remain distinguishable from infrastructure failures. Invalid registration data is returned to the form, invalid credentials produce a login message, unauthenticated page access redirects to login, and rejected API mutations use an error status with explanatory text and current state where available.

7. Modules of the Project - Phase-wise explanation of design and execution of Modules

The first phase represented the domain using Java classes. User carries identity and contact fields used by registration, login, and the profile view. Cycle identifies a rental item and exposes its descriptive type, hourly rate, and available flag. Booking joins a user and cycle for a duration and booking date, and computes a total from the cycle rate and number of hours. Activity captures a timestamp, actor name, and event message for the recent-activity panel.

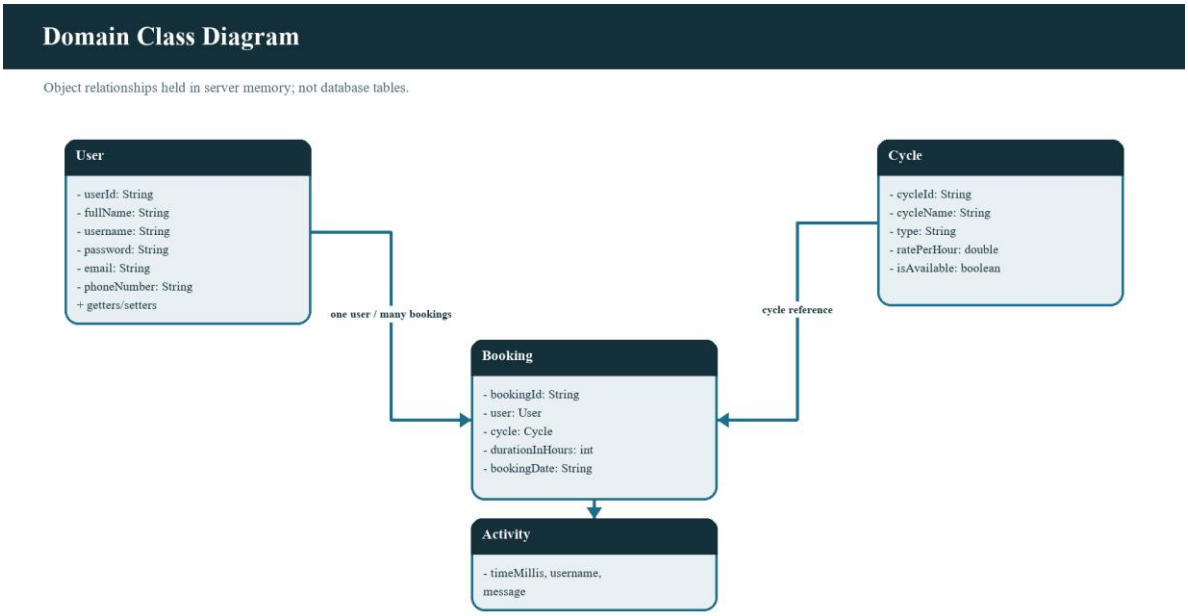


Figure 5. Domain classes and conceptual associations used by the project.

These models provide names and boundaries for the values used throughout the program. Rather than pass independent strings and numbers through every component, services can return a Cycle or Booking object and the JSON builder can read fields from those types. The model layer is not a database mapping: the classes contain ordinary Java fields, constructors, accessors, and selected behavior, and no persistence annotations or mapping library appears in the source.

The class diagram presents the conceptual relationships that can be observed in the code. A Booking has references to one User and one Cycle. Activity retains an actor label and event description; it is not a durable audit record. This model is sufficient to explain the demonstration, while a future persisted design would need explicit identity, normalization, constraints, and retention policies.

Phase 1: object responsibilities and data lifecycle

User is a mutable account model with a no-argument constructor, a full constructor, accessors, and a toString representation that omits the password. Cycle is also mutable; its constructor sets availability to true and service operations switch that flag during booking and cancellation. Booking stores the linked objects and supports a defensive zero-cost result if its cycle is missing or duration is nonpositive. Activity is final and its fields are final, so activity entries are immutable after construction.

The data lifecycle starts when AppListener initializes the web application. It creates one user service and one booking service and stores each in the context. BookingOperationsImpl creates the six initial cycles in its constructor. RegisterServlet creates User objects when valid registration requests arrive. The booking service creates Booking and Activity instances after a successful operation. Requests retrieve snapshots or filtered lists from the service and serialize them for the browser.

No state is restored after a server restart. The user map is rebuilt empty and the cycle list is reseeded. The booking list, end-time map, and activity feed likewise start empty. This lifecycle is important to the system model because an in-memory object relation is not evidence that a permanent record exists.

The model classes carry the values needed by the running services and presentation layer, while avoiding a claim that the objects are ORM entities. AppListener initializes application-wide service instances when the web application starts. Those services seed cycles and hold users, active bookings, and activity in collections.

Phase 2: operation interfaces

The service interfaces define operations separately from their concrete collection-based implementations. UserOperations exposes registration and authentication. BookingOperations exposes cycle listing, booking, cancellation, a rider's bookings, end-time lookup, and recent activity. Servlets use these contracts rather than embedding all business checks in page code.

The split supports a clear request path and makes the implementation responsibilities inspectable. RegisterServlet and LoginServlet depend on user operations. ApiServlet depends on booking operations. AppListener constructs the concrete implementations once and shares them through the ServletContext. A later implementation could retain the service contract while adding a repository-backed implementation, although the current project does not include that alternative.

The interface boundary is not dependency injection through a framework. The listener performs the setup explicitly, using string context keys. This approach is simple for the practical project and easy to follow, but the keys are not compile-time checked as a typed registry would be. Startup configuration and the correct use of those keys therefore remain important integration details.

UserOperations and BookingOperations expose actions the rest of the application needs without requiring callers to know the internal collection structure. The implementations own validation, lookup, and state changes. Servlets depend on these operations instead of duplicating booking rules in request handlers.

Phase 2: registration rules and service checks

UserOperationsImpl stores accounts in a ConcurrentHashMap keyed by trimmed username. registerUser rejects a null User, checks the full-name format, validates a username with at least four allowed characters, requires a password with at least six characters and both letters and digits, checks an e-mail pattern, and requires exactly ten digits for a phone number. A duplicate key is rejected before the new account is added.

Authentication requires non-null username and password. The username is trimmed for lookup and a missing user or wrong password produces the same invalid-credentials message. This avoids revealing which part of a login failed. The implementation compares the submitted password with the stored string because password hashing is not part of this academic prototype; the report marks that as a security limitation rather than recommending plaintext storage.

The user service synchronizes registration as an operation and uses a concurrent map for storage. Correctness depends on validation and insertion remaining an operation that cannot admit the same username twice. A future database-backed service would also require a unique constraint, because an application-level check alone does not protect concurrent registrations across multiple processes.

The registration path checks the submitted values before adding a user and prevents a username already present in the in-memory collection from being registered again. The Servlet handles request-specific concerns such as password confirmation and redirects; the service handles account validity and storage.

Phase 2: booking rules and synchronized state

BookingOperationsImpl owns the seeded fleet, active booking list, end-time map, activity feed, and next booking counter. Public methods that read or mutate shared booking state are synchronized. Before several operations, releaseFinishedBookings checks expiry and updates the related lists and availability flags. getAllCycles returns copies of Cycle objects so rendering code does not receive direct references to the original fleet list entries.

bookCycle validates the user, duration, cycle identifier, and availability. On success it records a formatted local booking date, sets availability false, creates a booking identifier, stores an end timestamp based on the injected clock, and adds an event. cancelBooking validates user presence, finds the requested booking, verifies owner username, then restores availability and removes the booking and timer. These checks centralize policy in the service.

The constructor accepts a LongSupplier clock, with the normal constructor supplying the system clock. This is a useful test seam because a test could advance time without waiting a real rental duration. The repository does not include a test suite exercising that seam, so the report describes testability as a design capability rather than a reported automated pass.

BookingOperationsImpl owns the authoritative cycle lookup, availability check, duration validation, total calculation, and booking creation. Synchronization protects these linked reads and writes from concurrent requests in the same process. Expired reservations are cleaned as state is accessed.

Phase 2: domain errors and rule traceability

InvalidUserDataException, AuthenticationException, and BookingException identify expected service failures. The names communicate the kind of operation that failed and let each Servlet respond in the right context. Registration catches invalid-user errors and forwards them to the registration page; login catches authentication errors and forwards them to login; ApiServlet catches booking errors and serializes the operation message with the refreshed state.

Rule	Enforcement point	Expected result
Name, username, email, phone	UserOperationsImpl	Reject invalid account data
Password confirmation	RegisterServlet	Reject mismatch before registration
Login credentials	UserOperationsImpl	Reject invalid credentials
Hours 1-12	BookingOperationsImpl	Reject unsupported duration
Cycle exists and is free	BookingOperationsImpl	Reject unknown or booked cycle
Cancellation owner	BookingOperationsImpl	Reject a different rider
Session present	Servlets and protected JSPs	Require authentication

This is preferable to treating every rejected action as a successful operation. A duplicate username is not inserted; a wrong password does not create a session; an occupied cycle is not booked; and a booking owned by another rider is not cancelled. A useful review traces each user-visible rule from input field or API parameter to its server-side check and then to the response behavior.

The exception design focuses on expected business conditions. It does not claim to be a complete operational logging strategy. Unexpected failures such as a missing context service or runtime infrastructure error need appropriate container logging and error handling. A production system would also define privacy-safe logs and avoid recording passwords or unnecessary personal data.

The application defines focused exception types for invalid user data, authentication, and booking failures so callers can distinguish expected domain outcomes. Servlet handlers translate those failures into redirects, form messages, or API status codes.

Phase 3: page architecture and shared fragments

The web application contains public login and registration JSPs and authenticated dashboard, bookings, and profile JSPs. Shared JSP fragments supply the header, navigation, footer, guard, and SVG symbols. This reduces repeated markup and gives the authenticated area a consistent navigation structure. The web.xml welcome-file-list points the application context to login.jsp.

The page split follows rider tasks. Registration collects account data. Login accepts credentials and displays server messages. The cycle dashboard supports catalogue discovery and booking. The bookings page shows the current user's active reservation and activity feed. The profile page presents account details from the session. The access guard prevents the authenticated pages from being used without a session user.

JSPs are server-rendered, but the dashboard view is subsequently updated by JavaScript. This creates a mixed interaction model: form-based transitions for account workflows and client-side API calls for live rental operations. The project uses shared includes rather than a larger template engine. This is adequate for a compact Tomcat application, while a larger product might adopt a component or server-side rendering framework.

The authenticated pages reuse JSP fragments for the document head, navigation, footer, session guard, and SVG sprites. Shared fragments keep the active navigation state and common assets consistent without repeating their markup in every page.

Phase 3: dashboard rendering and client state

dashboard-view.js converts a JSON state object into strings for cycle cards, filter chips, booking rows, and activity events. Dynamic text is escaped before it is inserted into HTML. It provides helpers for money formatting, remaining-time formatting, progress, relative event times, cycle categories, and type-specific SVG symbols. The module has no direct page access for rendering calculations and exposes functions that can be loaded by Node for isolated tests.

dashboard.js owns the DOM elements and interactions. It loads current state, stores the selected category and duration choices, compares generated HTML or booking signatures before redrawing, attaches booking and cancel actions, and updates the remaining-time indicators once per second. It also makes periodic state requests and reflects connection state in the Live/Reconnecting status pill.

The separation between pure rendering and event handling reduces coupling. It also creates useful test boundaries: formatting and escaping helpers can be tested independently from fetch and DOM behavior. However, the supplied repository does not include a configured JavaScript test runner or actual test source files. The architecture enables testing but is not evidence that a particular test suite was executed.

dashboard-view.js formats server data into cards, booking rows, status text, and activity entries, escaping dynamic text before inserting it as HTML. dashboard.js owns DOM events, filter state, duration choices, polling, countdown updates, and short-lived feedback messages.

Phase 4: Java web application architecture

The Servlet/JSP runtime separates request handling from the service layer. AppListener is annotated as a web listener and creates the user and booking operations when the context initializes. It also removes them from the context during shutdown. The public Servlet routes are annotated in source: registration, login, logout, and the API path. This avoids a large manual servlet mapping table in web.xml.

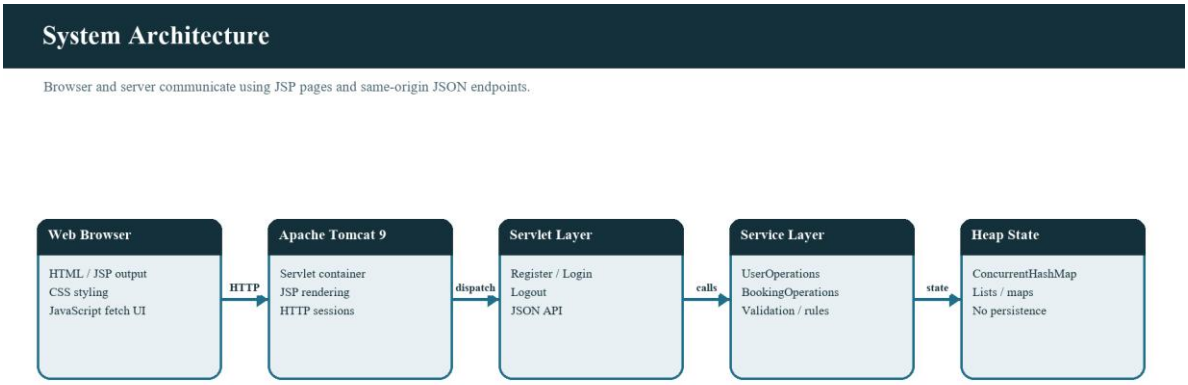


Figure 6. Application architecture based on the project source and Tomcat runtime.

The diagram shows the browser at the presentation boundary, Tomcat hosting JSP and Servlets, the service contracts and implementations in the middle, and Java objects in server memory at the data boundary. The session carries the authenticated user identifier/object for the rider's requests. No database layer appears because the source has none.

The architecture is simple and direct. It is suitable for demonstrating a complete request pipeline and for explaining the project's runtime dependencies. It is not designed for horizontal scaling, service discovery, asynchronous messaging, or durable recovery. A future design could preserve the API and user-facing pages while replacing collection storage with persistent repositories and adding deployment-level security controls.

Phase 4: Servlet responsibilities

RegisterServlet handles GET by redirecting to register.jsp and POST by reading the submitted profile fields, trimming text values, comparing password confirmation, constructing a User, and delegating registration. It assigns an identifier from an atomic counter. On success it redirects to login with a registration indicator; expected validation failures are forwarded back with an error message.

LoginServlet handles GET by choosing the dashboard or login page based on an existing session. Its POST validates required parameters, delegates credential authentication, invalidates a prior session, creates a new session, stores the user, and sets a thirty-minute inactivity timeout. LogoutServlet invalidates the current session where one exists and returns to a public page.

ApiServlet handles GET state requests and POST mutation requests. It verifies the API path, requires the current authenticated user, checks the mutation header, parses duration, dispatches book or cancel, and builds JSON replies. The service owns the booking rules; the Servlet owns HTTP details such as method, path, status, encoding, and response format.

Servlets translate HTTP details into service calls. Registration and login process form parameters and choose a forward or redirect; logout invalidates the session; ApiServlet reads JSON-action parameters, checks authentication, and returns structured JSON with appropriate status codes.

Phase 4: JSON state and privacy-aware activity

StateJson.build obtains cycle copies, the current user's bookings, and recent activity from BookingOperations. It calculates the available count and builds a state object containing server time, limited user display data, cycle identifiers/names/types/rates/availability, that rider's bookings, and recent activity. It reports activity actors as "You" for the current user and "Another rider" for other users.

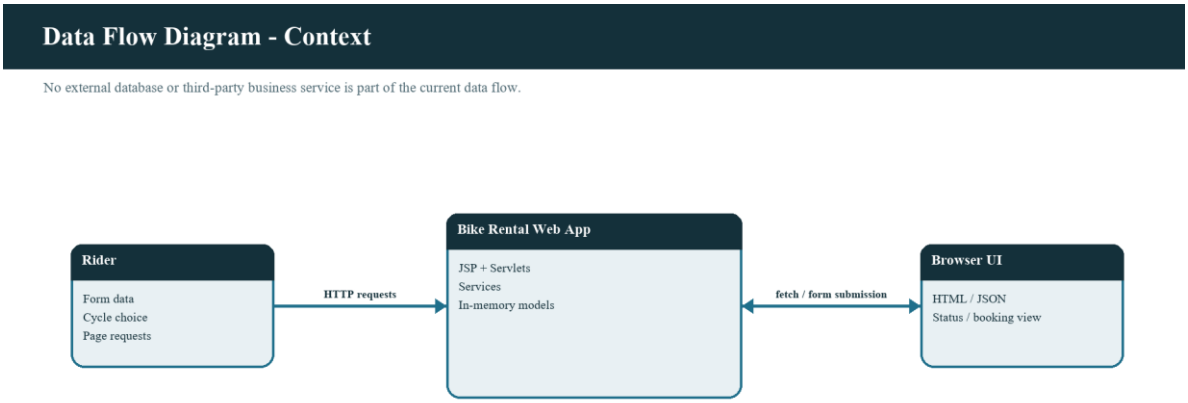


Figure 7. Context data flow between rider, browser, and application.

Json.quote is used for text values so string content is represented as JSON rather than concatenated raw. StateJson uses a Locale-independent decimal format for rates and totals. The response includes server time so the browser can estimate clock offset when displaying countdowns. The time remaining is a view calculation; the service's end-time map is the source used for state.

The feed is bounded to the latest eight events by BookingOperationsImpl. That limit controls in-memory growth and avoids showing an unlimited activity history. It does not provide durable audit retention. The generic actor label reduces username exposure in the feed, although the current user's own activity is labeled directly as "You." Data minimization and retention would need further design for a production service.

StateJson builds a snapshot from the current user, cycle list, bookings, activity, and server time. The activity feed uses generic labels for other riders instead of exposing their usernames. Client rendering escapes text fields before inserting them into the page.

Phase 4: state data-flow details

The level-one data-flow view separates authentication, page handling, API operations, and in-memory state. A registration or login request follows the account path. A signed-in dashboard requests a snapshot containing the catalogue, its own active rentals, and the activity feed. A booking or cancellation request follows the mutation path and changes service state only after validation.

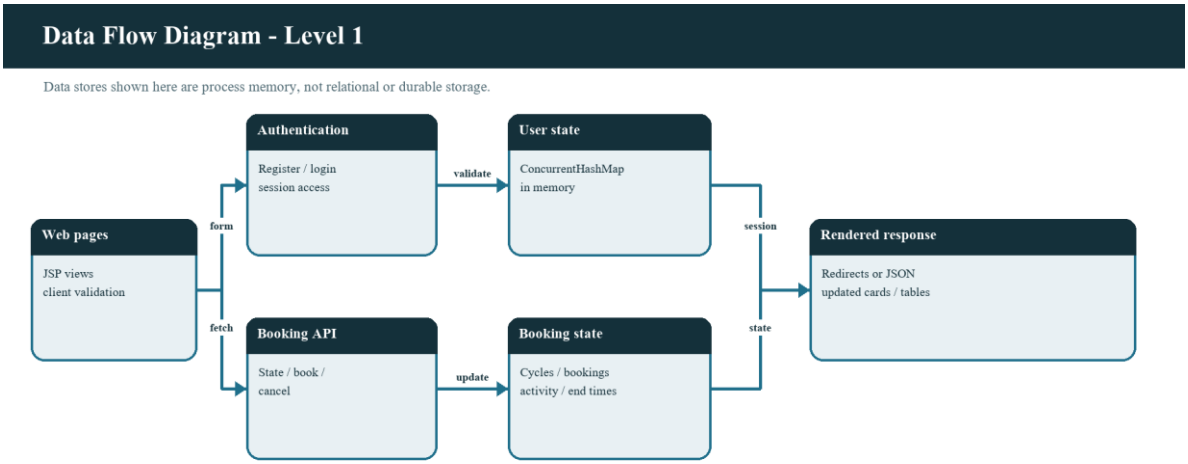


Figure 8. Level-one application data flow and collection-backed state.

The user service and booking service are separate context attributes. The user map stores registered accounts. The booking service stores the fixed fleet, active booking objects, end timestamps, and a bounded recent-event list. The session stores the authenticated user for request access control. These stores are process memory and are cleared when the web application is restarted.

The drawing uses data-flow terminology to make paths legible, not to assert the presence of a deployed database. There is no SQL data store behind the service. The cycles shown in the catalogue are seeded from Java constructors, and current bookings are collection entries. A later persisted implementation would need a different data-store node and transaction rules.

Operational lifecycle, synchronization, and limits

Within the running instance, synchronized booking operations serialize access to shared list and map state. The service checks and changes the availability flag while holding the same operation lock, which prevents simultaneous requests in that process from both booking the same cycle. Cycle listing returns copies to reduce the risk that a presentation consumer changes the original fleet objects directly.

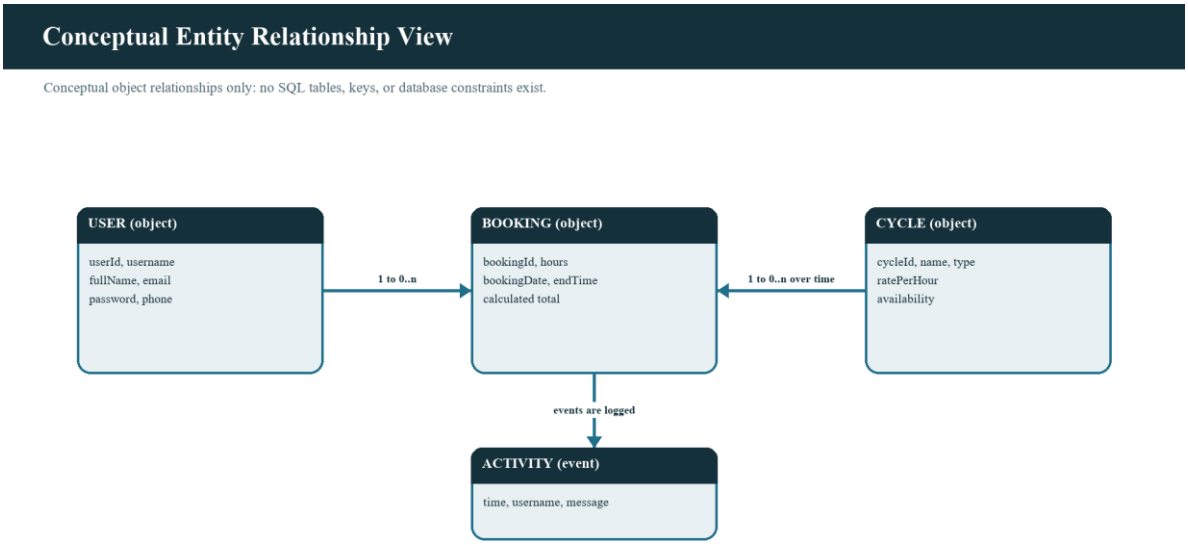


Figure 9. Conceptual object relationships; this is not an implemented database schema.

This is an in-process consistency measure only. If two application instances started with separate memory, they would not share a lock or current booking list. A production deployment would move the invariant into a shared transactional data store or another coordination mechanism and would define behavior for simultaneous booking transactions across servers.

The class relationship drawing can help identify which records need durable relationships if the system grows. A relational design could represent users, cycles, bookings, and activity as tables with keys and constraints, but this report does not supply or claim such a schema. The object relation diagram is conceptual and should be read with the memory-only lifecycle caveat.

The listener initializes the application services at startup, and shutdown discards their process-local collections. Booking synchronization covers requests handled by one service instance; it does not provide durable writes, recovery, or coordination with another process.

Java implementation concepts and design rationale

The preceding module discussion describes the responsibilities of the project components. This section examines the Java mechanisms that make those responsibilities work together, including service contracts, exception flow, container-managed lifecycles, synchronized state changes, time representation, and JSON construction. These are implementation choices visible in the source, not claims that the application uses a framework or a persistent data platform.

Typed service contracts and model boundaries

UserOperations and BookingOperations are Java interfaces. They name the operations required by the rest of the application without exposing the collections used by UserOperationsImpl and BookingOperationsImpl. RegisterServlet and LoginServlet call the user contract, while ApiServlet calls the booking contract. This is a practical form of programming to an interface: a servlet can depend on the operation it needs rather than know how a user map or booking list is organized. AppListener still constructs the concrete classes directly; there is no framework-based dependency injection or alternative storage implementation in the current project.

The model package gives these operations typed values. A User carries identity and contact fields, a Cycle carries catalogue attributes and availability, a Booking links a user and cycle to a duration, and Activity represents one feed item. Constructors and accessors make the fields available to the service and presentation layers. Booking.calculateCost places the rate-times-duration calculation with the booking data rather than duplicating that formula in a servlet. Activity is immutable after creation because its class and fields are final. User, Cycle, and Booking are mutable, which is convenient for this small application but means callers must respect the service as the owner of booking state.

This object model is not an object-relational mapping. The project has no persistence annotations, repository framework, or database identity map. A Booking holds direct references to its User and Cycle while the process is running. If a database is added later, the service boundary can remain useful, but a repository would need to load related records and preserve referential integrity instead of relying on these in-memory object references.

Checked exceptions as business outcomes

InvalidUserDataException, AuthenticationException, and BookingException extend Exception, so callers must either handle them or declare them. This checked-exception design makes expected business rejection visible in the method signatures: registration can fail validation, authentication can fail to match an account, and booking or cancellation can violate a domain rule. The service reports why an operation was rejected; it does not decide whether the result should become a page or an HTTP response.

The Servlet layer performs that translation. Registration and login forward the request to the corresponding JSP with an error message, while successful form submissions redirect to another page. The API converts booking failures into a conflict response with JSON that the dashboard can display. This separation keeps HTTP details out of the service implementation. It also distinguishes a rejected request from an infrastructure problem: the current handlers catch the expected domain exception types, rather than treating every possible runtime failure as a normal booking rejection.

The account checks are split according to their meaning. RegisterServlet compares the password confirmation and normalizes text fields because those are properties of the submitted form. UserOperationsImpl validates the account values and checks username uniqueness because those rules apply whenever the account service is called. BookingOperationsImpl validates duration, cycle existence, availability, and booking ownership at the point where it controls the booking state. This placement means that hiding a button or changing browser code cannot replace the server-side rule.

Servlet lifecycle and scope of shared objects

The application uses the Servlet API supplied by Tomcat 9. AppListener is annotated with @WebListener and receives contextInitialized when the web application starts. It creates one UserOperationsImpl and one BookingOperationsImpl, then stores them as attributes of ServletContext. On shutdown it removes the attributes. The listener therefore acts as the composition point for the application: it connects concrete Java services to the Servlet runtime without adding a separate dependency-injection library.

The ServletContext is scoped to the deployed web application, not to an individual request or rider. A servlet retrieves the shared services from that context, while Auth.currentUser uses request.getSession(false) to look for an existing HttpSession without silently creating one. Request parameters belong to one HTTP exchange, the session holds the authenticated user across requests, and the context holds services shared by requests. Keeping these scopes distinct is important: storing the whole booking state in a session would give different riders separate views of availability, while storing an authenticated user in application scope would mix identities.

Servlet annotations declare the URL mappings, and doGet or doPost provide the method-specific entry points. The handlers read form parameters, set request character encoding where needed, invoke service operations, and set redirects, forwards, status codes, and response content types. The separation is intentionally direct and appropriate for a small Servlet/JSP exercise. A larger application could centralize route concerns or use a framework, but it would still need to preserve the same ownership and validation rules.

Thread safety and the limits of synchronization

Tomcat can process multiple requests at the same time. The listener shares a single booking-service object within the application, so concurrent requests can reach the same lists and maps. BookingOperationsImpl synchronizes its public operations that read or change booking state. In bookCycle, the availability check, cycle status change, booking insertion, end-time insertion, and activity recording occur while holding the same object monitor. A second request for the same cycle must wait and then observes the changed availability. This makes that transition indivisible to other synchronized methods on this service instance.

The account service uses a ConcurrentHashMap keyed by username and synchronizes registration so checking for a duplicate and inserting the account remain one operation. AtomicInteger supplies user and booking identifier counters that can be incremented safely by concurrent request threads. These mechanisms address different concerns: the map supports concurrent lookup, the synchronized method protects a multi-step registration rule, and the atomic counter protects increments. None of them makes the data durable.

The guarantee is deliberately limited to one BookingOperationsImpl instance. A second Tomcat process would have its own lists and lock, so the two processes could each consider

the same cycle available. A database-backed version would need a transaction or uniqueness constraint shared across instances. Even within one process, StateJson obtains cycles, a rider's bookings, and activity through separate service calls. Each call is protected, but the combined JSON document is not a single locked snapshot. If strict cross-field consistency becomes a requirement, the service should expose one snapshot operation that reads the related values under one lock and returns an immutable state object.

The service returns copies of the cycle objects and creates new lists for collection results, reducing direct access to its internal collections. These are shallow copies in some cases: a returned Booking still refers to the same User and Cycle objects. Current request handlers serialize or display those values rather than changing them, so the existing flow stays within its intended ownership boundary. Immutable response objects would make that boundary stronger if additional callers or concurrent features are introduced.

Time representation and deterministic behavior

BookingOperationsImpl accepts a LongSupplier clock. Its default constructor supplies System.currentTimeMillis, while the second constructor can receive another source of milliseconds. The service records booking end times as epoch milliseconds and computes the expiry comparison using that numeric value. It also uses Instant, ZoneId.systemDefault, and LocalDateTime to produce a readable booking date. This separates the decision about whether a booking has expired from the formatting of a date shown to a person.

The injected clock is a useful test seam. A service-level test can provide a controllable clock, create a booking, advance the supplied time beyond the end timestamp, and call a service read that invokes releaseFinishedBookings. The test can then check that the booking has been removed, the cycle is available, and a finished-ride activity item was recorded without waiting for real hours to pass. This describes a feasible test method; no test execution or successful result is claimed in this report.

Expiry is evaluated lazily when selected service methods read the state; the project does not schedule a background timer. This is adequate while the application is being actively used, but an expired booking may remain in memory until a later read triggers cleanup. The displayed booking date uses the server machine's default time zone, whereas the end time is an absolute timestamp. A persistent service used across regions should choose and document a consistent time-zone policy, normally storing instants in a standard form and applying a user-facing zone only for display.

JSON construction, encoding, and value safety

StateJson builds the dashboard response from typed model values. It includes a server timestamp, limited user display fields, cycles, the current rider's bookings, and a bounded activity feed. Json.quote encodes string values by escaping quotation marks, backslashes, control characters, and HTML-sensitive characters; numeric values and Boolean availability are emitted as JSON values rather than quoted strings. StringBuilder assembles the response, and Locale.ROOT is used when formatting rates and totals so that a machine's decimal separator does not depend on its regional settings.

The servlet sets the API response to application/json, UTF-8, and no-store. These headers define how the browser reads and caches the response; they do not themselves validate the contents. HTML and JSON are different output contexts. The project has a separate Html.escape helper for HTML text, and the dashboard view also escapes dynamic values before creating markup. Correct context-specific encoding is necessary because JSON

quoting alone is not a general-purpose HTML sanitizer, and HTML escaping should not be substituted for JSON serialization.

The JSON is assembled manually rather than through a serialization library. This keeps the payload structure visible and avoids an additional dependency, but it also means each new field must be added with the correct type, quoting, and delimiter rules. A response DTO plus a maintained JSON library would be a reasonable later change if the API grows. The client and server would then benefit from an explicit versioned schema and tests for missing, additional, and malformed values.

Design implications and measured extension points

Several implementation choices are intentionally simple and have clear upgrade paths. The service interfaces can be retained while storage is moved behind repository implementations; the service rules can be tested independently from HTTP; the injected clock can support expiry tests; and the state response can be represented as an immutable snapshot. These are extension points, not already implemented features. Any storage migration must make availability checking and booking creation atomic in the shared store, rather than relying on the current process-local monitor.

The cost calculation currently uses double and the supplied hourly rates are whole rupee amounts. This is adequate for the displayed demonstration values, but decimal floating-point is not an ideal basis for financial accounting because some decimal fractions cannot be represented exactly in binary. If discounts, taxes, partial hours, or payment records are introduced, monetary values should use a defined currency scale and a decimal type such as BigDecimal, with rounding rules documented and tested. No payment, tax, or invoice calculation is part of the current application.

The table below summarizes the Java mechanisms in relation to their actual use and their boundary. Reading the mechanisms together shows that the project favors explicit, inspectable code: contracts describe operations, the Servlet container supplies lifecycle objects, and synchronized service methods protect in-memory transitions. Production capabilities such as durable transactions, password hashing, and multi-instance coordination remain proposed improvements rather than properties of these mechanisms.

Java mechanism	Use in this project	Boundary to retain
Interfaces and typed models	UserOperations, BookingOperations, User, Cycle, Booking, Activity	Contracts do not add persistence or interchangeable implementations by themselves.
Checked domain exceptions	Validation, authentication, and booking rule failures	They represent expected business outcomes, not all infrastructure failures.
Synchronized methods and concurrent collections	Protect shared in-memory state and account registration in one application instance	No coordination across multiple Tomcat processes or durable transaction.
AtomicInteger and LongSupplier	Generate process-local IDs and provide a replaceable service clock	Counters reset on restart; deterministic testing is enabled but not supplied.
ServletContext and HttpSession	Share service instances application-wide and retain the signed-in user per rider	Context attributes are not a persistent store or a typed dependency registry.
Json.quote and StringBuilder	Create the JSON state consumed by the dashboard	Manual response construction requires field-level encoding and schema tests.

8. Source Code (Only functional code structure)

The project root separates Java source, the web application, deployment instructions, and report assets. Java code is organized under com.bikerental. The webapp directory contains JSP views, JavaScript, CSS, the deployment descriptor, and shared JSP fragments. build.bat compiles the source and copies the webapp directory to the Tomcat application context.

Path	Functional responsibility
src/com/bikerental/model	User, Cycle, Booking, Activity
src/com/bikerental/exception	Expected validation, authentication, and booking failures
src/com/bikerental/service	Operation interfaces and in-memory rules
src/com/bikerental/servlet	HTTP route handlers and application lifecycle
src/com/bikerental/util	Session lookup, HTML/JSON helpers, dashboard state
webapp/*.jsp	Public and authenticated user views
webapp/js, css	Browser interaction, rendering, and presentation
webapp/WEB-INF	Deployment descriptor, classes, shared JSP fragments

The source structure below identifies functional code only, as requested by the supplied template. It is a compact map rather than a full source listing. The Java source packages are the editable implementation; WEB-INF/classes contains compiled outputs generated by the build. The source should remain the authoritative material when reviewing or changing the application.

Project_Report/Diagrams contains the implementation-oriented diagrams used in this document, and screen shot contains the six supplied browser screenshots. These are supporting assets, not Java application code. The phase-outline crop is intentionally not used as a report screenshot or application feature.

This section maps the working source rather than reproducing every file in the document. The Java source under src is the editable server implementation; webapp holds JSP views, shared fragments, CSS, and browser scripts. WEB-INF/classes contains compiled output created by the build.

Java packages and responsibility map

The model package contains the objects passed between service and presentation layers. The exception package gives expected failures distinct types. The service package contains UserOperations and BookingOperations contracts along with collection-backed implementations. The servlet package contains AppListener, RegisterServlet, LoginServlet, LogoutServlet, and ApiServlet. The utility package contains authentication lookup, JSON generation, HTML escaping, and state serialization helpers.

This organization keeps request parsing outside core booking rules. For example, ApiServlet reads a form parameter and interprets the path; BookingOperationsImpl checks cycle and booking invariants. LoginServlet understands the HTTP session; UserOperationsImpl compares submitted credentials against the current account store. StateJson converts model and service values into the state representation consumed by JavaScript.

The JSP files map to rider tasks: login.jsp and register.jsp are public; dashboard.jsp presents cycles; bookings.jsp presents active rentals; and profile.jsp displays account/session details. Shared fragments under WEB-INF/jspf hold navigation, page structure, guards, and SVG symbols. This separation makes it easier to find the functional surface corresponding to a user-visible behavior.

The model package contains User, Cycle, Booking, and Activity; service contains the account and reservation operations; servlet contains the request endpoints and lifecycle listener; util contains authentication, HTML, and JSON helpers; exception contains domain failures.

Build, deployment, and request routes

build.bat expects TOMCAT_HOME to point at a Tomcat 9 installation. It verifies that the Servlet API JAR exists, compiles Java source with javac --release 17, outputs classes to WEB-INF/classes, and copies the web application into webapps/BikeRental. The operator should confirm the target path before running the script because it removes the existing deployed application directory before copying the new one.

The main browser routes include login.jsp, register.jsp, dashboard.jsp, bookings.jsp, and profile.jsp. Registration and login POST to their mapped Servlets. The JSON endpoint paths are GET /api/state, POST /api/book, and POST /api/cancel. The deployment descriptor sets login.jsp as the welcome page and a thirty-minute session timeout with an HttpOnly session cookie.

If deployment fails, check the Java version, Servlet API path, Tomcat path, compile output, Tomcat logs, and application context separately. A successful copy does not prove that the server started correctly, and a successful home page response does not by itself prove every authenticated booking workflow. Diagnostic steps should identify which layer failed before changing code.

build.bat compiles against the configured Tomcat Servlet API and copies the webapp directory into the BikeRental deployment context. The welcome page is login.jsp. Account forms use the registration and login Servlet mappings, while the live dashboard reads state and submits booking or cancellation actions through the API.

9. Features of the Project

The application features are presented through the six user-supplied screenshots and the behavior visible in the source. The screenshots cover the public login and registration forms, the cycle catalogue, an empty bookings view, an active booking, and the profile page. Together they show the navigation and page states a rider encounters; they are evidence of the supplied interface, not a claim that every possible browser or error case was tested.

The catalogue image shows category filters and cycle cards with type and price. The bookings images illustrate both the empty state and a live booking with total cost, countdown, progress indicator, and cancel action. The profile image shows account and session details. The registration and login images show labels, inputs, and the route between public account pages.

Screenshots preserve their captured application content. Some sample account or session values are visible because they are part of the supplied evidence. Their presence does not make those values durable project data. The implementation uses in-memory services, so the screenshots should be understood as captured examples of rendered state.

The feature set combines account access, a type-filtered cycle catalogue, hourly cost previews, server-validated bookings, countdowns, cancellation, activity, and profile/session information. The following screenshots are evidence of selected rendered states.

Screenshot: login page

The login screenshot shows the public sign-in view. Its left panel presents a short product message and a bicycle-wheel illustration; the right panel contains a username field, password field, submit button, and a link to create an account. A status message is visible in the captured state. The layout visually separates application branding from the credential form and uses the same shared color system seen on authenticated pages.

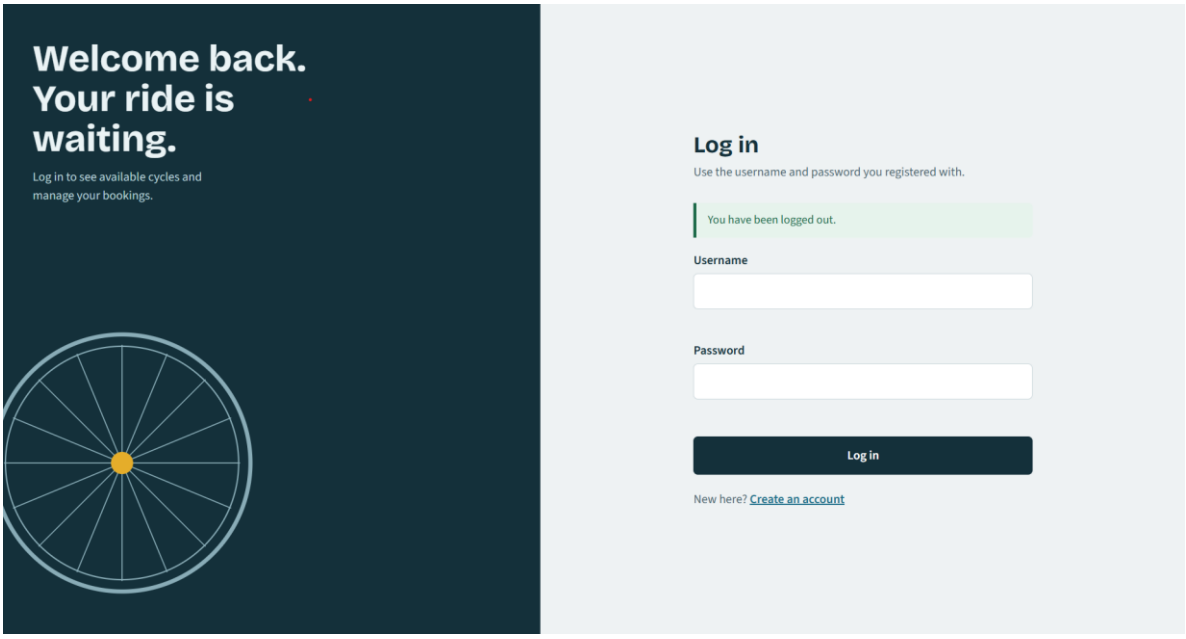


Figure 10. Supplied screenshot of the Bike Rental login page.

The login JSP posts the username and password to LoginServlet. The browser script provides field-level feedback, while the Servlet rejects missing values and the user service checks the credential pair. Successful authentication redirects to the dashboard and establishes a new thirty-minute session. Invalid credentials are returned to the login view as a message. The screenshot records one rendered state; it does not expose a credential value or establish a security certification.

The view separates product identity from the credential task, helping a first-time visitor locate the sign-in fields and registration route. The form posts to the login Servlet; browser validation improves feedback, but authentication is performed by the server service.

Screenshot: registration form

The registration screenshot presents the new-account form with fields for full name, username, e-mail, phone, password, and confirmation. Hints explain the username, phone, and password formats, and the account-creation action is paired with a route back to login. This view makes the required data visible before a visitor submits the form.

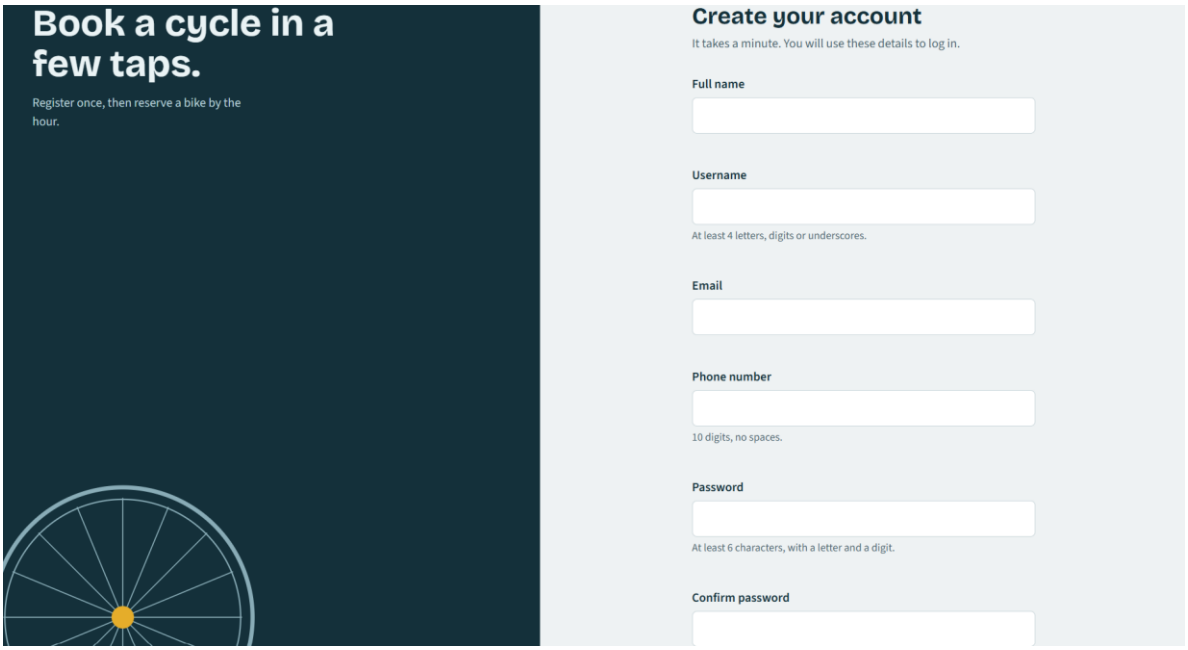


Figure 11. Supplied screenshot of the create-account registration page.

The browser can highlight format errors near the corresponding field, but the server repeats validation in UserOperationsImpl. RegisterServlet compares the password confirmation, trims text fields, assigns a user identifier, and delegates registration. The service rejects missing or malformed values and duplicate usernames. On success the user is redirected to login; on expected failure the form is forwarded with an error. The account is stored in memory rather than a database.

The form exposes the required account fields together and gives short format hints beside the relevant controls. The confirmation value is compared on submission, while the service checks the account fields and username uniqueness.

Screenshot: cycle catalogue

The catalogue screenshot shows the authenticated navigation, a live indicator, an availability summary, category filters, and three cycle cards in the visible viewport. Each card includes cycle art, a name, category, hourly price, a duration selector, calculated total, and a Book button. The filters include All, Mountain, Hybrid, Road, and Electric, allowing a rider to narrow the current display by type.

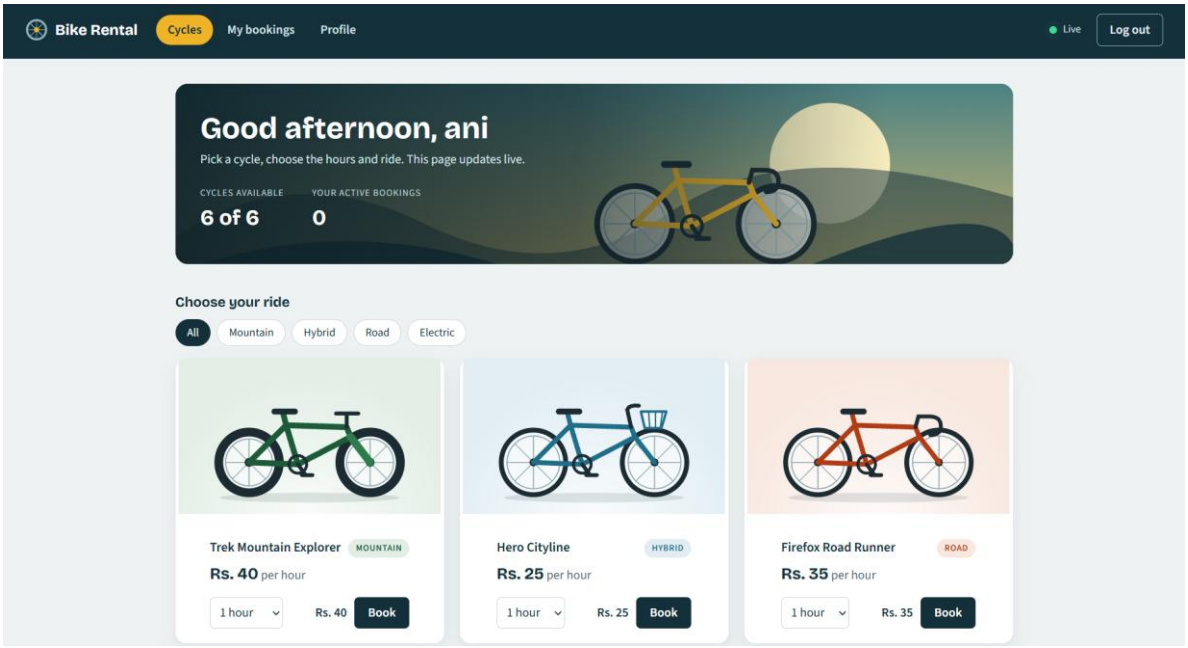


Figure 12. Supplied screenshot of the authenticated cycle catalogue and booking controls.

The server state contains six total cycle entries, while the screenshot viewport shows only those cards that fit on the displayed screen area. The dashboard uses a four-second polling interval to request state. When a cycle availability value changes, the client can update its card and statistics. The client cost is a preview; the service validates the submitted cycle and duration and computes the booking total from the stored rate.

The catalogue combines availability totals, category filters, cycle art, hourly prices, duration selectors, and booking actions. A filtered card is still backed by the same server inventory as an unfiltered view.

Screenshot: empty bookings and activity

The empty-bookings screenshot shows the My bookings view when the current user has no active reservation. The main panel communicates that there is no booking yet and offers a route back to cycles. The adjacent activity panel can still show recent events from the fleet, with other riders displayed generically. The summary at the top reports active bookings and the number of cycles currently available.

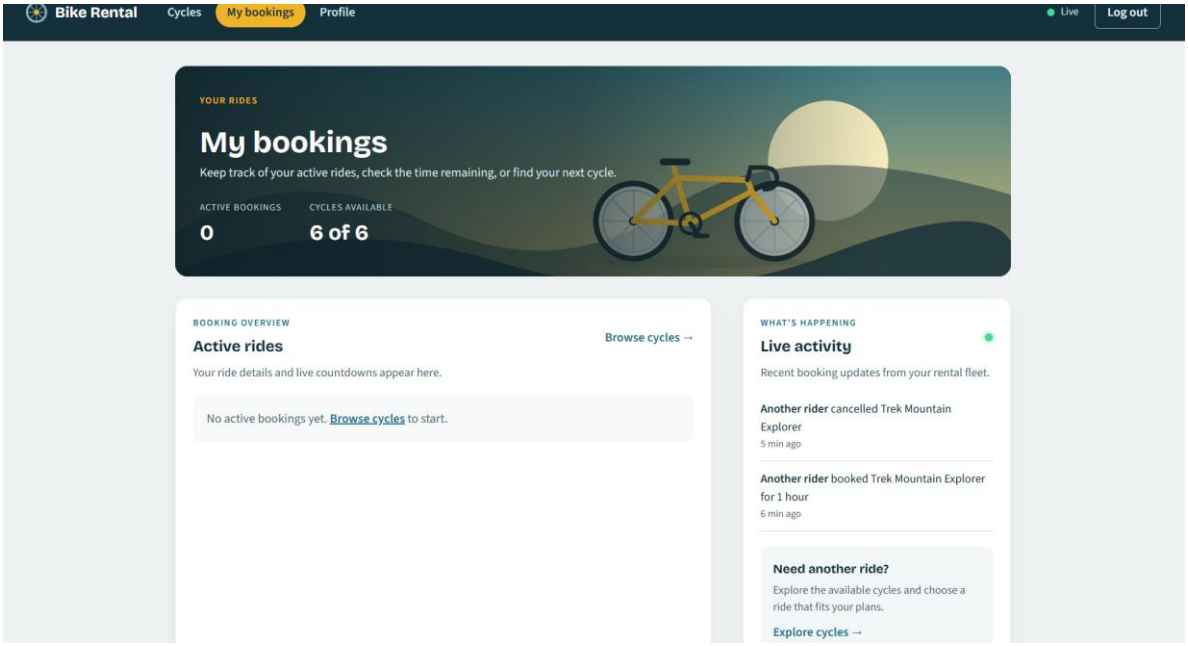


Figure 13. Supplied screenshot of the empty bookings view and recent activity.

The empty state is a deliberate user-interface condition rather than an error. StateJson returns an empty booking array for a rider who has no active rentals, and dashboard-view.js converts that state into an explanatory message. The activity feed is separate from the rider's own booking list, so it can contain events from other users while preserving generic identity labels. The feed is bounded in memory to the latest eight activity entries.

An empty personal booking list is a valid state and is presented separately from the fleet-wide activity stream. The activity area can show recent events even when the current rider has no reservation, using generic labels for other users.

Screenshot: active booking and countdown

The active-booking screenshot shows one current reservation in a table with booking identifier, cycle, duration, total cost, remaining time, progress bar, and a Cancel button. The activity panel records the rider's booking event and other recent fleet events. The top summary also shows how an active booking reduces the number of cycles available.

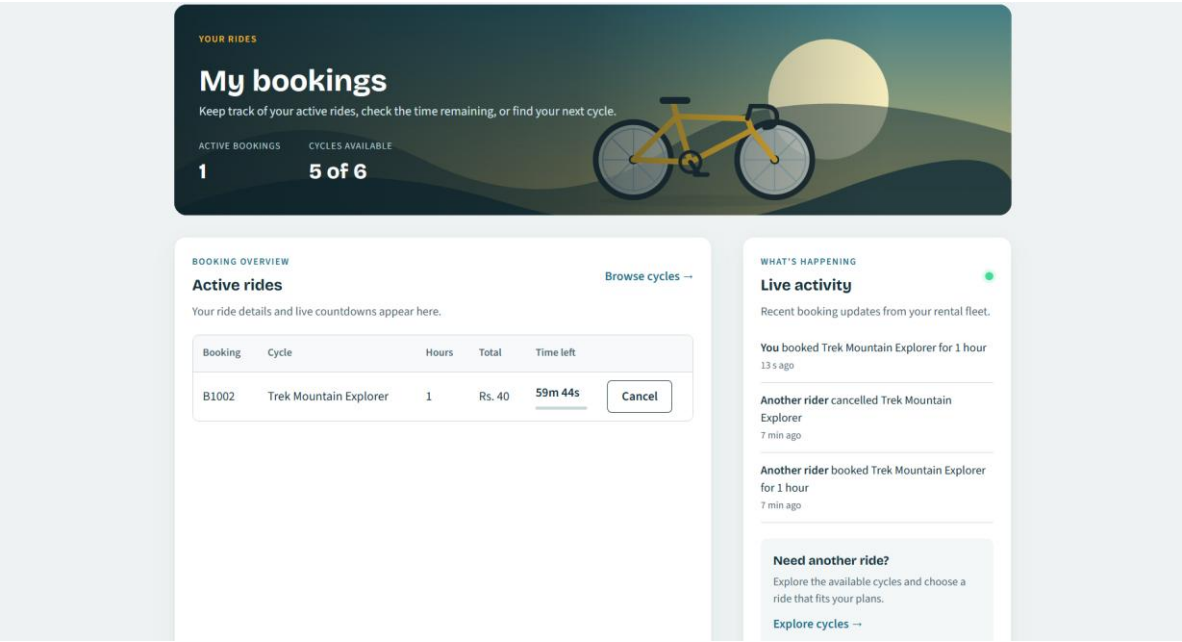


Figure 14. Supplied screenshot of an active booking, countdown, and cancellation control.

The server includes each booking end time and sends server time in the JSON state. The browser estimates the time offset and updates countdown text every second without redrawing the entire table for every tick. If a countdown reaches its end, the dashboard requests a fresh server snapshot so the service can release the expired booking. Cancellation requires a second confirmation click in the interface, while the service independently verifies ownership.

The booking row presents an identifier, cycle, duration, total, end-time countdown, progress indicator, and cancellation action. The countdown is a browser display derived from the server end time; the service determines when an expired reservation is released.

Screenshot: profile and session view

The profile screenshot shows the signed-in user's full name, username, e-mail, phone, user ID, session sign-in time, session timeout, and active-booking count. A shared navigation bar allows the rider to return to cycles or bookings and provides a logout control. The screen is read-only in the supplied application.

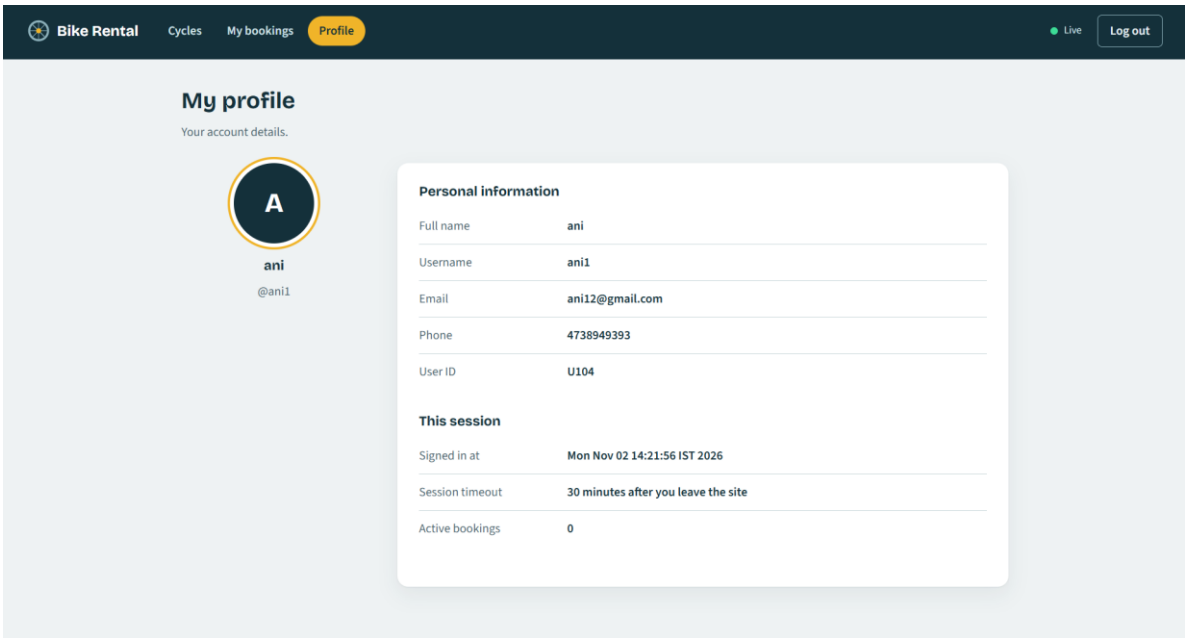


Figure 15. Supplied screenshot of the rider profile and session details.

The profile view uses the current session user and current booking state; it is not an independently editable profile store. The session timeout is thirty minutes of inactivity. Password is not a profile display field, and the User.toString method also omits its password. However, the current service implementation holds passwords as plain strings in memory for authentication, which remains a limitation. Screenshot account details are captured examples and should be reviewed before external publication.

The profile is a read-only summary of the signed-in user and current session. It displays account attributes and booking count, while omitting the password. Session timing helps explain why a rider may need to log in again after inactivity.

Dashboard interaction and live feedback

The dashboard separates static page structure from frequently changing state. On initialization it requests state from the API, renders cycle types and cards, then continues polling about every four seconds. The client compares generated HTML or booking signatures before redrawing. This helps preserve a selected duration and avoids unnecessary recreation of controls while the countdown text updates separately.

Booking and cancellation submit same-origin fetch requests with URL-encoded form data and the request header expected by ApiServlet. A successful response contains a message and new state; a rejected request can return a conflict response with explanatory text and current state. Toasts provide short feedback and are removed after a timeout. The live pill switches to a reconnecting state after a failed request and returns to live after recovery.

The front end improves perceived immediacy, but it is not the source of truth. A user can edit requests or bypass the page. The server validates duration, availability, account session, and booking owner. This division is a key feature of the implementation and a useful boundary to preserve in any future interface rewrite.

The page polls state on a fixed interval and updates only the regions that need to change, preserving a selected filter or open duration choice where possible. A one-second timer updates countdown text independently of the slower state poll.

Feature verification and honest test reporting

The project package includes browser screenshots showing rendered application states and source code showing the associated Servlet and service paths. A disciplined verification plan should cover valid and invalid registration, duplicate username, correct and incorrect login, unauthenticated protected-page access, cycle display, valid booking, booking an unavailable cycle, viewing a booking, owner cancellation, non-owner cancellation, and expiry after the selected time.

The code provides expected behavior for these cases, but the source package does not include JUnit test sources or a configured JavaScript test suite. Therefore this report does not claim a complete automated pass. Screenshot evidence demonstrates the captured UI states, while code inspection supports the documented validation path. A reviewer should distinguish those evidence types instead of reading an expected outcome as an executed test result.

For repeatable future testing, the injected clock in BookingOperationsImpl can be used to advance time without waiting. Service tests can assert state transitions and exceptions; servlet tests can verify HTTP status and session behavior; and browser tests can verify filtering, countdown display, and two-step cancellation. Test data should use clearly synthetic account values and should not rely on an external live rental service.

Useful acceptance checks include registration boundaries, duplicate usernames, valid and invalid login, unauthenticated page access, filtering, booking an available cycle, rejecting a conflict, owner cancellation, non-owner cancellation, and expiry. The project has supplied screenshots and source evidence, but no configured automated test suite is included.

Representative acceptance test cases

The following cases are derived from the supplied servlet and service rules. They are a test plan, not execution results; the project package contains no automated test suite or recorded

test run.

ID	Scenario	Expected behavior from source	Status
TC-01	Register with valid profile details and matching passwords	Account is added to the in-memory user service; browser is redirected to login with a registration notice.	Not executed
TC-02	Submit an invalid e-mail or phone number during registration	Server-side validation rejects the request and returns an explanatory message to the registration page.	Not executed
TC-03	Register a username that already exists	Duplicate username is rejected; existing account remains unchanged.	Not executed
TC-04	Log in with an incorrect username/password pair	Authentication fails and the login page displays an error.	Not executed
TC-05	Open a protected page or API without an authenticated session	JSP guard redirects to login; API returns HTTP 401 JSON.	Not executed
TC-06	Book an available cycle for 1 to 12 hours	Server creates one booking, marks the cycle unavailable, and calculates rate multiplied by duration.	Not executed
TC-07	Book with a duration outside 1 to 12 hours or an unavailable cycle	Booking service rejects the request; API reports a conflict response for booking-rule failures.	Not executed
TC-08	Cancel another rider's booking	Ownership check rejects cancellation; the other rider's booking remains active.	Not executed
TC-09	Allow a booking to reach its end time	A later service read releases the cycle and adds a finished-ride activity item.	Not executed
TC-10	Log out and revisit a protected page	Session is invalidated and the protected page requires login again.	Not executed

10. Future Enhancements

The highest-value extension is durable storage. A relational database could store accounts, cycles, bookings, and activity so their lifecycle survives Tomcat restarts. A schema would need primary keys, a unique username constraint, a relation from booking to user and cycle, a rule that prevents more than one active booking for a cycle, and fields for booking start and end times. Migration scripts would make schema changes repeatable.

The service interfaces provide a potential seam for introducing repository-backed implementations, but the current code does not include a DAO or JDBC layer. A migration should preserve service-level validation while moving persistence and transactional concurrency control to a shared database. Booking a cycle should be one transaction that checks availability and creates the reservation under a lock or unique active-booking constraint.

Persistence also raises retention decisions. Expired and cancelled bookings may be retained for history, while the activity feed may need a separate bounded or time-based retention policy. Sensitive account fields should be minimized. Backups, recovery, schema migration testing, and database connection configuration become operational responsibilities once data is made durable.

Enhancement work should follow the current service boundaries rather than bypass them. Durable storage is the clearest next step, followed by stronger credential handling, repeatable tests, and operational observability. Each proposal should be tied to a limitation in the running implementation and introduced with acceptance criteria.

Security, maintainability, and deployment roadmap

Credential storage should be upgraded before the service is used beyond a controlled demonstration. Passwords should be stored using a modern salted password-hashing function, never as plaintext. Account recovery and password changes require verification and token lifecycle rules. Production deployment should use HTTPS and secure session-cookie attributes, and should use an explicit CSRF protection design for state-changing requests rather than relying solely on a client-supplied header.

Maintainability improvements could externalize the Tomcat path, separate build and deploy stages, provide a repeatable clean deployment, and add logging for unexpected failures without recording secrets. An administrator role could manage cycle inventory and rates with authorization checks. Automated service and servlet tests should verify validation rules, ownership, expiry, and concurrency. Browser tests should include keyboard access, screen-reader labels, responsive layouts, and supported browser versions.

Operational scaling requires replacing process-local synchronization with shared transaction semantics. Observability should report application health and request failures without exposing personal data. Each future feature should be introduced with a measurable requirement and tests, so that a larger architecture does not undermine the simple rider flow already demonstrated.

A persistence migration should preserve server-side validation and make the availability check plus booking insert atomic. Password hashing, secure session-cookie settings, HTTPS, and explicit CSRF protection should be addressed before public deployment. Repeatable service and browser tests can then verify ownership, expiry, concurrency, and accessible interactions.

11. Conclusion

The Bike / Cycle Rental Booking System demonstrates a complete academic web-application path from a rider’s browser to Java server-side behavior. The model classes represent users, cycles, bookings, and activity. Service interfaces and implementations validate account and booking operations. Servlets handle registration, login, logout, and JSON API requests. JSP and shared fragments produce the views, while JavaScript adds client validation, category filters, live state updates, cost previews, countdowns, and feedback.

The supplied screenshots make the principal application states visible: public login and registration, the cycle catalogue, an empty bookings view, an active booking, and a profile/session view. The project’s diagrams explain the workflow, request sequence, architecture, data movement, and object relationships. Together, source and visual material show how the functional modules meet at the server boundary.

The implementation is intentionally bounded. User and booking data are held in memory, passwords are not hashed, inventory is fixed, there is no payment or database integration, and synchronized state is limited to one application instance. Those limitations do not remove the value of the integration exercise; they define the work required for a persistent and hardened service. The project therefore meets its purpose as a practical demonstration while providing a concrete foundation for future development.

The result is a working academic integration exercise with a clear boundary: it demonstrates server-side account and booking workflows while keeping state in memory and inventory fixed. The report?s strongest evidence is the connection among source responsibilities, request diagrams, and captured interface states.